☐ **Project Name**

PlugPorch

☐ **Team Member Names & Email Addresses**

John Suli, xsuli@seas.upenn.edu
Jia Wu, jiaywu@seas.upenn.edu
Thien Tong, thient@seas.upenn.edu
Vincent Luu, luuvince@seas.upenn.edu

☐ **Brief Introduction**

- What was the problem?
  EV owners are limited to existing public and commercial chargers, which are not always available in every area of the city. In the unfortunate event that a driver becomes stranded far from a charging location, their options are often very limited.

- What was the team trying to accomplish?
  The solution is to build an app that connects EV owners with individuals who have home chargers and are willing to rent them out. This allows EV drivers to charge their vehicles at someone's garage or driveway, effectively turning private chargers into shared, rentable assets — much like how Airbnb transforms homes into temporary rentals.

☐ **Proposed Design Brief, high-level description of the solution**

- **User Experience**: New users start with a simple onboarding experience. After signing up using Clerk, they land on the home screen, which features a Google Map centered on their current location with nearby chargers highlighted, plus personalized recommendations and a quick view of recent bookings. They can also choose a full-screen map for easier browsing or an all listings screen to explore all available charger listings. Users can become hosts by setting up their own charging station and managing incoming requests. A profile screen provides access to transaction history, while real-time notifications keep users updated on booking statuses. Users can also chat with each other directly through in-app messaging.
- **Backend**: The backend stores user, host, booking, and listing information securely, integrates with third-party services for authentication (Clerk) and payment (Stripe), and supports real-time messaging between EV owners and hosts (using Socket Io).
- **Frontend**: The frontend was built using React Native (and Expo for file routing) that allows users to search, book, and manage charger rentals efficiently; message other users in real time; and make secure payments.

☐ **Impact**

- How does this influence performance, security, and other system aspects?
  Security: Authentication via Clerk and secure storage of sensitive user credentials. Only hosts can manage listings, and only users involved in a booking can access its details.
  Payment: Sensitive data like payment info is handled by Stripe on the backend which provides an invoice that is stored in our database.
  Messaging: Implemented socket IO for real-time messaging.
  Reliability: Proper validation of bookings, payments, and notifications, combined with try–catch handling and database references ensures consistency across the platform.

☐ **Example Details to Include:**

- System Architecture – List components of the system (and/or provide a diagram of their relationship)

  Frontend: Home, Listings Map, All Listings, Host, Chat, Profile
  Backend:: handles User, Listing, Booking, Chat, Payment, Notification API endpoints and real time messaging with Socket Io,
  Database: MongoDB collections for users, bookings, listings, messages, notifications, payments, reviews
  External Integrations: Stripe, Clerk, Maps API

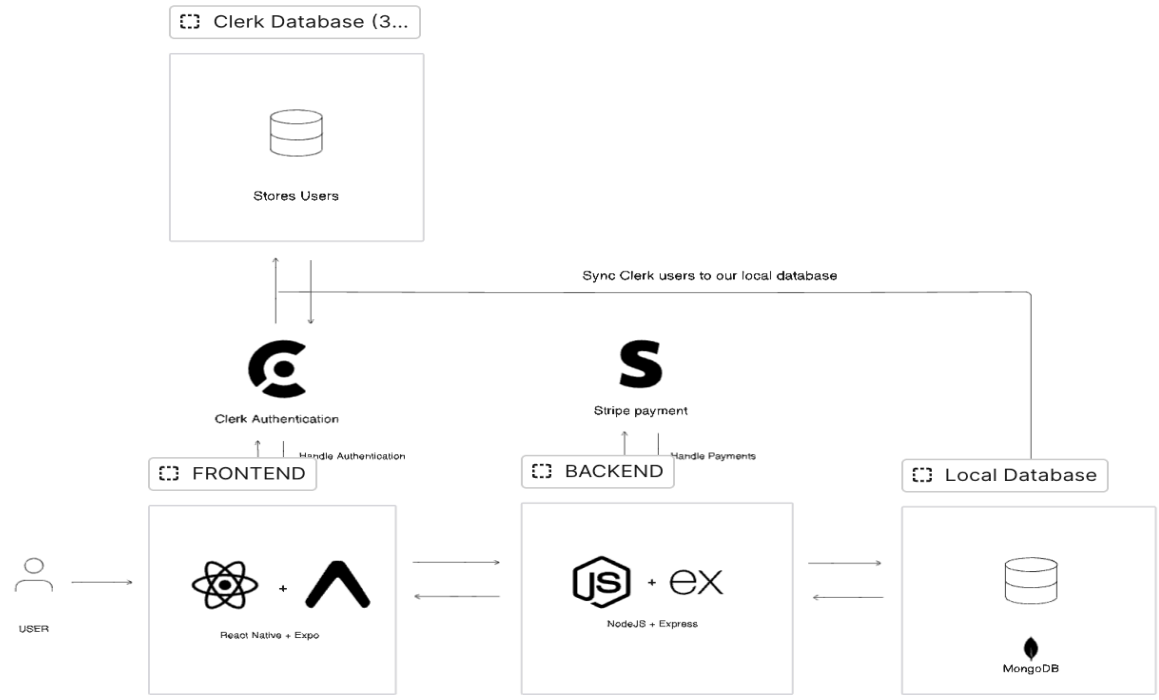

Figure 1. Overview of System Architecture.

- Data Model – How is data stored? What is the database schema?

  All data is stored in MongoDB in the following schemas:

- **User table**: id, email, clerkId, firstName, createdAt (Note: Users are also securely stored in Clerk's own database, which is synced with our local database)
- **Conversation table:** id, name, category, location, start_time, end_time
- **Booking table**: id, charger_listings_id, charger_listings_address, host_id, ev_owner_id, start_time, end_time, total_cost, status, payment_status, rating_by_driver, rating_by_host, battery_level, images
- **Listing table**: id, host_id, charger_type, power_output_kw, connector_type, address, latitude, longitude, availability_schedule, price_per_hour, min_price, images, is_active, instructions, created_at, updated_at
- **Message table**: id, conversation_id, sender, text, createdAt
- **Notification table**: id, user_id, message, read
- **Payment table**: id, receiver, payer, booking_id, amount, createdAt
- **Review table**: id, host_id, ev_owner_id, rating, comment, createdAt

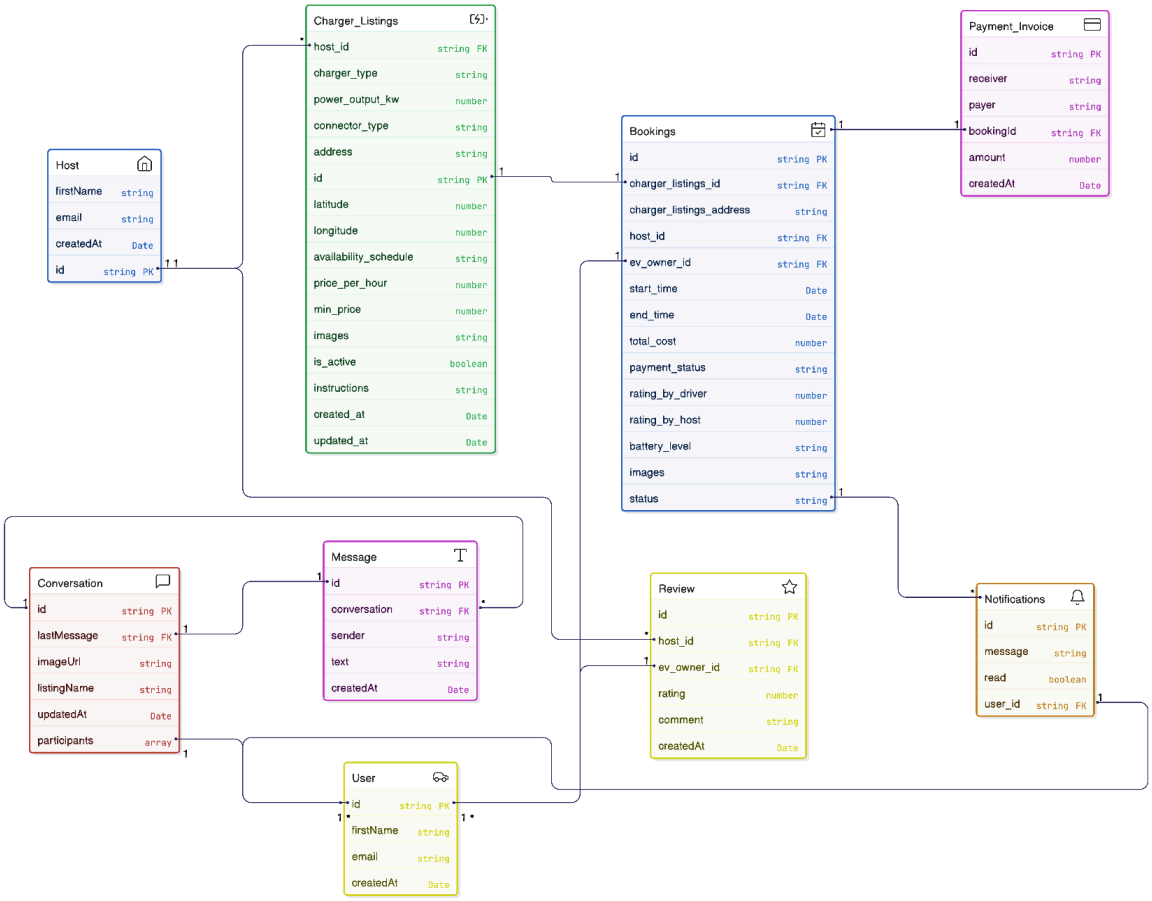

Figure 2. Entity relationship diagram.

- Interface/API Definitions

  1. User APIs
  POST /api/users/ — Create a new user
  PUT /api/users/ — Update user information
  DELETE /api/users/ — Delete a user
  GET /api/users/:email — Get user details by email

  2. Listing APIs
  GET /api/listings/ — Get all listings
  GET /api/listings/:id — Get listing by ID
  GET /api/listings/users/:hostId — Get listings by host ID
  POST /api/listings/ — Create a new listing
  PUT /api/listings/:id — Update a listing
  DELETE /api/listings/:id — Delete a listing

  3. Booking APIs
  GET /api/bookings/ — Get all bookings
  GET /api/bookings/:id — Get booking by ID
  GET /api/bookings/user/:userId — Get bookings for a specific user
  GET /api/bookings/host/:hostId — Get bookings for a specific host
  POST /api/bookings/ — Create a booking for a listing
  PUT /api/bookings/:id — Update a booking
  DELETE /api/bookings/:id — Delete a booking

  4. Conversation & Message APIs
  GET /api/conversations/ — Get all conversations
  GET /api/conversations/:userId — Get conversations by user ID
  GET /api/conversations/:conversationId/messages — Get messages in a conversation
  POST /api/conversations/ — Create a new conversation

  5. Notification APIs
  GET /api/notifications/ — Get all notifications
  GET /api/notifications/unread-count — Get unread notifications count
  POST /api/notifications/ — Create a new notification
  PUT /api/notifications/:id/read — Mark notification as read

6. Payment / Transaction APIs
POST /api/payment-sheet/ — Initiate checkout/payment sheet
POST /api/transaction/ — Create a payment transaction
GET /api/transaction/:userId — Get transactions for a user

7. Review APIs
POST /api/reviews/ — Submit a review

☐ **Reflections** - Each team member should type a 5 sentence reflection describing their experience with SPARC.

1. John - SPARC helped me learn time management and the logistics of building an app, including delegating tasks, merging branches in Git, and testing. The program gave me ample opportunity to build the app and served as a great motivator to learn a new framework. It allowed me to create something I can proudly add to my portfolio. Joining SPARC was a good decision because it provided motivation to continue working on the app even when I hit deadlocks. These challenges were made manageable with the support of team members and mentors, since building an app from scratch alone can be overwhelming.
2. Jia - SPARC allowed me to build a mobile app alongside a collaborative team. I had always wanted to build a mobile app, and SPARC provided me with the opportunity to collaborate with other students to create something I cared about. Working with others, I learned how to solve problems as a group, and communicate more effectively. From this experience, I also learned a new development framework and explored different platforms for testing. Overall, I really enjoyed the experience and am proud of the product we created in SPARC.
3. Thien - Participating in SPARC gave me the chance to work on a mobile app within a collaborative team. I learned how to break down complex problems, manage my time effectively, and contribute meaningfully to a shared project. Through this experience, I learned a new development framework, and gained practical experience in testing and debugging. Overall, I am proud of the app we built and grateful for the opportunity to learn and grow alongside my team.
4. Vincent - Working on SPARC taught me the importance of clear communication when working in a team. Having weekly check-ins kept us aligned when plans changed and enabled us to re-establish expectations. It also allowed us to ask help early and share short updates so that everyone can follow along with each other’s work. Finally, code reviews and tests together helped us catch issues early on. Overall, seeing our separate pieces click together into a working app inspires me to continue seeking opportunities to build software with a team.